

仿真环境下创建地图

1) 启动 roscore

```
gnome-terminal --title="roscore" -- bash -c "roscore"
```

等待 roscore 就绪

```
until rostopic list >/dev/null 2>&1; do sleep 0.2 done
```

2) 启动 Gazebo + robot

```
gnome-terminal --title="gazebo_start" -- bash -c "source ~/Gitkraken/ros1_slam/devel/setup.bash;  
roslaunch tri_steer_gazebo carto_test.launch"
```

等待 Gazebo 节点起来 (/gazebo/model_states 出现再继续)

```
until rostopic list 2>/dev/null | grep -q "/gazebo/model_states"; do sleep 0.5 done
```

3) 启动 teleop + robot

```
gnome-terminal -t "teleop_start" -- bash -lc "source ~/Gitkraken/ros1_slam/devel/setup.bash; roslaunch  
tri_steer_keyboard keyboard_tri_steer.launch"
```

4) 雷达点云反转成水平车体下的point-cloud2

```
gnome-terminal -t "laser to cloud" -- bash -lc "source ~/Gitkraken/ros1_slam/devel/setup.bash; roslaunch  
read_laser_data laser_to_cloud.launch; exec bash"
```

5) 启动cartographer建图节点

```
gnome-terminal -t "carto gmapping" -- bash -c "cd  
/home/action/Cartographer/Cartographer_Localization;source install_isolated/setup.bash;roslaunch  
cartographer_ros my_robot_2d.launch;exec bash"
```

保存创建的地图

6) 保存地图

```
cd /home/action/Cartographer/Cartographer_Localization source install_isolated/setup.bash rosservice  
call /write_state "{filename: '/home/action/carto_map.pbstream'"
```

7) 转换 pbstream → 栅格地图

```
roslaunch cartographer_ros cartographer_pbstream_to_ros_map -  
pbstream_filename=/home/action/carto_map.pbstream -map_filestem=/home/action/carto_map -  
resolution = 0.1
```

启动仿真环境下的导航

```
source ~/.bashrc set -e
```

1) 清理旧进程（允许失败）

```
rosnode kill -a 2>/dev/null || true killall -9 gzserver gzclient 2>/dev/null || true killall -9 roscore rosmaster  
2>/dev/null || true
```

```
yes | rosclean purge
```

```
sleep 2
```

2) 启动 roscore

```
#ros_term "roscore" "roscore" gnome-terminal --title="roscore" -- bash -c "roscore"
```

等待 roscore 就绪

```
until rostopic list 2>/dev/null 2>&1; do sleep 0.2 done
```

3) 启动 Gazebo + robot

```
#这里面需要将地图更改成构建好的地图 gnome-terminal --title="gazebo_start" -- bash -c "source  
~/Gitkraken/ros1_slam/devel/setup.bash; roslaunch tri_steel_gazebo sim_gazebo_rviz.launch"
```

等待 Gazebo 节点起来（/gazebo/model_states 出现再继续）

```
until rostopic list 2>/dev/null | grep -q "/gazebo/model_states"; do sleep 0.5 done
```

4) 启动 teleop + robot

```
gnome-terminal -t "teleop_start" -- bash -lc "source
~/Gitkraken/ros1_slam/devel/setup.bash; roslaunch
tri_steer_keyboard keyboard_tri_steer.launch"
```

5) 启动地图节点

```
gnome-terminal -t "map start" -- bash -lc "source ~/Gitkraken/ros1_slam/devel/setup.bash; roslaunch
robot_costmap map_deal.launch; exec bash"
```

6) 运动规划节点

```
gnome-terminal -t "motion Plan start" -- bash -lc "source ~/Gitkraken/ros1_slam/devel/setup.bash;
roslaunch robot_navigation motionPlan_sim.launch"
```

7) 雷达点云反转成水平车体下的point-cloud2

```
gnome-terminal -t "laser to cloud" -- bash -lc "source ~/Gitkraken/ros1_slam/devel/setup.bash; roslaunch
read_laser_data laser_to_cloud.launch; exec bash"
```

8) 局部规划PID算法*****

```
gnome-terminal -t "PID_control" -- bash -c
"source ~/Gitkraken/ros1_slam/devel/setup.bash; roslaunch robot_pid_local_planner
Omnidirectional_PID.launch; exec bash"
```

9) 启动并发布机器人真值里程计消息

```
gnome-terminal -t "truth Odom start" -- bash -lc "source ~/Gitkraken/ros1_slam/devel/setup.bash;
roslaunch robot_locatization truth_odometry.launch; exec bash"
```

10) 订阅控制消息

```
gnome-terminal -t "control start" -- bash -lc "source ~/Gitkraken/ros1_slam/devel/setup.bash; roslaunch
tri_steer_drive bringup.launch use_teleop:=true; exec bash"
```

仿真环境下的导航重定位

常用命令

#查看tf树 `roslaunch rqt_tf_tree rqt_tf_tree`

查看tf位姿变化 `roslaunch tf_echo map base_footprint`

